

## Session 3 · Deck 3.A3

# Euler Equations, Time Iteration, and What Is Still Missing

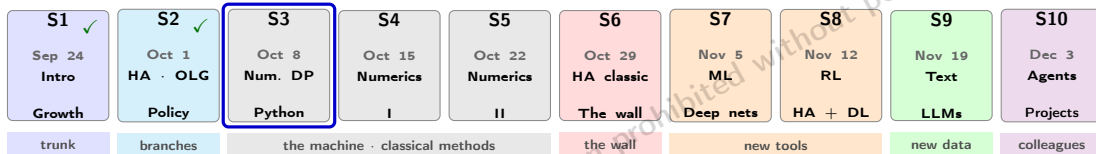
Zhigang Feng

Thursday, October 8, 2026

*Build it on paper → solve it on the machine → push it to the frontier with AI*

# Where We Are

you are here: 3.A3 ▼



*Build it on paper → solve it on the machine → push it to the frontier with AI*

**Block A:** 3.A1 what it means to solve a model · 3.A2 value function iteration on a grid · 3.A3 Euler equations and time iteration · 3.A4 the worked example, end to end.

Hands-on, 3.B5:

1 Version 0: grid VFI against the closed form

2 Data → parameters:  $g, n, \alpha, \delta, \beta$

3 The calibrated economy, solved and checked

4 Test the model: untar-geted ratio, Euler residual

# One Model, Two Methods, One Judge

Same model, same grid, same shock, same closed form to grade against, and both policies graded on the same object:  $\max |c/c^* - 1|$ .

|                              | VFI                  | Time iteration                         |
|------------------------------|----------------------|----------------------------------------|
| Iterations                   | 272                  | <b>17</b>                              |
| Wall time                    | <b>0.04 s</b>        | 0.59 s                                 |
| Max policy error             | $5.4 \times 10^{-3}$ | <b><math>6.7 \times 10^{-6}</math></b> |
| $\max \log_{10} \mathcal{E}$ | -1.8                 | <b>-3.8</b>                            |

**Sixteen times fewer iterations and eight hundred times more accurate** — and seventeen times slower.

Every one of those four numbers is explained by *one* design change: iterate on the policy and solve an equation, instead of iterating on the value and searching for a maximum.

# Why Leave the Value Function at All?

Three reasons, and the first two are visible in 3.A2's staircase:

1. **The policy is the object you want.** VFI drives  $v$  down to whatever tolerance you ask for, and then hands back a policy good to two digits. You paid for precision in the wrong function.
2. **The policy bends less.**  $v$  inherits the curvature of  $u$  ( $B \ln k$ , steep near zero);  $g = \alpha \beta z k^\alpha$  bends far less. A gentler function is cheaper to represent on a grid.
3. **No maximization.** Each point needs the solution of one scalar equation, not a search over  $N$  candidates.

## What you give up

The Euler equation requires **differentiability** and an **interior** optimum. VFI needs neither — it will happily solve a problem with discrete choices, kinks, or a binding constraint, which is exactly where the Euler road ends.

So this is not a replacement. It is a second instrument, with different failure modes — which is what makes the comparison worth something.

# The Euler Equation Is Also a Functional Equation

From the first-order and envelope conditions of 1.B2,

$$u'(c_t) = \beta \mathbb{E}_t [u'(c_{t+1}) R(k_{t+1}, z_{t+1})],$$

with the gross return  $R(k', z') = \alpha z' k'^{\alpha-1} + 1 - \delta$ . Write the policy as  $k' = g(k, z)$  and substitute. Consumption today is  $c = zk^\alpha + (1 - \delta)k - g(k, z)$ , and consumption tomorrow uses  $g$  again:

$$u'(zf(k) - g) = \beta \mathbb{E} [u'(z'f(g) - g(g, z')) R(g, z')].$$

One equation, one unknown — and the unknown is a function that appears composed with itself.

## The same two problems

**Represent**  $g$  finitely: values on a grid as before, now with linear interpolation between them.

**Kill the residual:** but now by *root-finding*, not by maximizing.

Decisions 1, 2 and 5 of 3.A1 are unchanged. Decision 3 is now needed —  $g(g)$  lands off the grid — and decisions 4 and 6 have moved.

# Time Iteration: The Operator

Given a guess  $g_n$  for *tomorrow's* policy, define  $g_{n+1} = Kg_n$  pointwise: at each  $(k, z)$ , find the  $k'$  that solves

$$u'(zf(k) - k') = \beta \mathbb{E} \left[ u'(z'f(k') - g_n(k', z')) R(k', z') \right].$$

- The left side is **increasing** in  $k'$ : saving more costs more today.
- The right side is **decreasing** in  $k'$  under standard conditions.
- So they cross **once**. The root exists and is unique — before any numerical method is chosen.

## Read the name literally

$g_n$  is the policy used *one period later*. Iterating  $K$  is solving a finite-horizon problem of growing length — “time” iteration.

Which also tells you where to start:  $g_0 \equiv 0$  (eat everything) is the last period of life, and the iterates then increase.

# Time Iteration: The Algorithm

**Input:** grid  $\{k_i\}$ , chain  $(z_j, \Pi)$ ; guess  $c_0 = y/2$ ;  $\text{tol} = 10^{-8}$ .

```
1: repeat
2:   for each grid point  $(k_i, z_j)$  do
3:      $y \leftarrow z_j k_i^\alpha$  ▷ resources
4:      $r(c) \equiv u'(c) - \beta \sum_m \Pi_{jm} u'(\hat{c}_n(y - c, z_m)) R(y - c, z_m)$ 
5:      $c_{n+1}(k_i, z_j) \leftarrow$  the root of  $r$  on  $(0, y)$  ▷ brentq
6:   end for
7: until  $\max |c_{n+1} - c_n| < \text{tol}$ 
8:  $g(k, z) \leftarrow zk^\alpha - c(k, z)$ 
```

$\hat{c}_n$ :  $c_n$  interpolated linearly at  $k' = y - c$ .  $R(k', z') = \alpha z' k'^{\alpha-1}$ , since  $\delta = 1$ .

On our grid that is  $200 \times 2 = 400$  scalar root-finds per sweep, and **6,800** in total across 17 sweeps.

## Two lines are new

**Line 4** needs  $c_n$  between grid points: Decision 3 now matters.

**Line 5** replaces the max by a root-find: Decision 4 has moved. Why the bracket  $(0, y)$  cannot fail is the next frame.

# The Black Box in the Middle

`brentq` is handed a bracket  $[a, b]$  on which the residual changes sign, and returns the root. Today we take that on trust, and we are entitled to: we *proved*, when we wrote the operator down, that the residual is strictly monotone in  $c$  and changes sign on the bracket, so it crosses zero exactly once, so a bracketing method cannot fail.

That is the whole reason this course does theory before numerics. **The existence-and-uniqueness argument is what licenses the black box.**

## What is inside it

Bisection is safe but slow; Newton is fast but needs a derivative and can fly off; the secant method splits the difference. Brent's method combines bisection with the secant step and inverse quadratic interpolation, and keeps the bracket.

We build each of them, and their convergence rates, **next week**.

Until then the rule is: a black box is acceptable exactly when you can state the condition under which it is guaranteed to work.

# Convergence Without a Contraction

VFI converged because  $T$  is a contraction — Blackwell gave us monotonicity *and* discounting, and Banach did the rest.  $K$  has no Bellman equation behind it, so that argument is unavailable.

The repair is monotonicity, and it reaches further than the contraction did:

- **Coleman (1990, 1991):** in the standard optimal-growth setting,  $K$  is **monotone**, and iterating it from a suitable guess converges to the unique equilibrium policy.
- **Greenwood–Huffman (1995):** the same argument in economies with externalities or distortions — where no planner's problem exists, and so no contraction to appeal to.

Coleman (1990, 1991); Greenwood & Huffman (1995).

## Why the second one matters

Taxes, externalities and market power break the welfare theorems. There is then *no* value function to iterate on — but the equilibrium Euler equation still holds, and  $K$  still converges.

This is the road that survives into the models of Session 2. It is why time iteration, not VFI, is the backbone of heterogeneous-agent and optimal-policy computation.

# Monotonicity Instead of Contraction

Start at  $H^0(K, z) = 0$  and define  $H^{j+1}(K, z) = x$  as the solution of

$$u'(F(K, K, z) - x) = \beta \mathbb{E} \left[ u'(F(x, x, z') - H^j(x, z')) \cdot F_1(x, x, z') \right],$$

where the second argument of  $F$  is the *aggregate* capital the household takes as given. The argument runs in three steps:

1.  $H^{j+1} \geq H^j$  whenever  $H^j \geq H^{j-1}$ : the operator is **monotone** (why: next frame).
2.  $H^j(K, z) \leq F(K, K, z)$ : the sequence is **bounded** by feasibility.
3. A monotone bounded sequence converges; equicontinuity makes the limit continuous.

## Then check it is an equilibrium

Convergence is not the end. You must still verify that the limit satisfies the *equilibrium* Euler equation once the consistency condition  $k = K$  is imposed.

Practical consequence: start from  $g_0 \equiv 0$  and **watch the iterates increase**. If they ever fall, the monotonicity hypothesis is violated somewhere and the guarantee is gone.

# Why Time Iteration Is Monotone

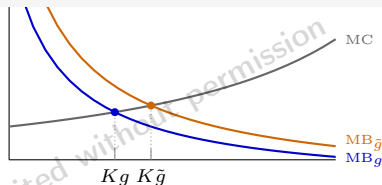
**Monotone** means *order-preserving*: if  $g \leq \tilde{g}$  at every  $(k, z)$ , then  $Kg \leq K\tilde{g}$  at every  $(k, z)$  — Blackwell's first condition, now for  $K$ .

$Kg(k, z)$  is the  $k'$  where the marginal cost of saving today meets its marginal benefit tomorrow:

$$\underbrace{u'(zf(k) - k')}_{\text{MC}(k') \text{ rises}} = \underbrace{\beta \mathbb{E}[u'(z'f(k') - g(k', z'))R(k', z')]}_{\text{MB}_g(k') \text{ falls}}.$$

**Raise  $g$  to  $\tilde{g}$ .** Tomorrow consumes less at every  $k'$ , so  $u'$  is higher ( $u$  is concave) and  $\text{MB}_{\tilde{g}} \geq \text{MB}_g$  everywhere. MC does not move, so the crossing moves **right**:  $K\tilde{g} \geq Kg$ .

**In words:** if your future self saves more, it consumes less, so a unit saved today is worth more, and you save more too. No value function is used, so the argument survives the previous frame's distortions.

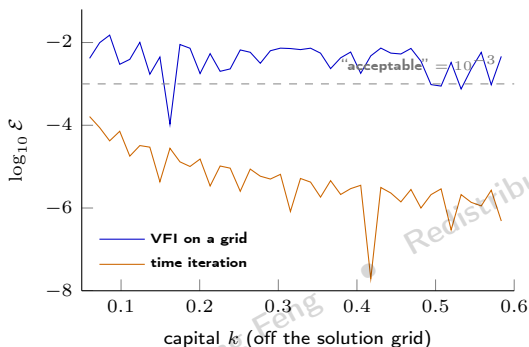


savings  $k'$ ;  $g \equiv 0$ ,  $\tilde{g} = Kg$ , output 1

## Our model, exactly

With  $u = \ln c$ ,  $\delta = 1$ , saving the share  $s$  maps to  $s' = \frac{\alpha\beta}{1-s+\alpha\beta}$ , increasing in  $s$ . From  $s_0 = 0$ : 0.255, 0.315, 0.333,  $\dots \rightarrow \alpha\beta = 0.342$ , the savings rate with  $n$  periods left. From our code's  $c_0 = y/2$  the shares fall: 0.406, 0.365,  $\dots$

## Grading Both, on Points Neither One Solved



The unit-free residual of 3.A1, at 997 capital levels chosen to fall *between* the solution grid points (plot: low  $z$ ; table: both states).

|                                         | VFI                  | TI                   |
|-----------------------------------------|----------------------|----------------------|
| log <sub>10</sub> of mean $\mathcal{E}$ | -2.40                | -4.92                |
| worst log <sub>10</sub> $\mathcal{E}$   | -1.78                | -3.75                |
| worst residual                          | $1.7 \times 10^{-2}$ | $1.8 \times 10^{-4}$ |
| true policy error                       | $5.4 \times 10^{-3}$ | $6.7 \times 10^{-6}$ |
| ratio                                   | $3.1 \times$         | $27 \times$          |

**Santos (2000), in practice:** the residual exceeds the error it certifies, for both methods.

VFI's worst case is a **1.7% consumption mis-take**: seventeen times 3.A1's "acceptable" 0.1%.

# VFI or Time Iteration — and Why You Write Both

|                   | Value function iteration                    | Time iteration                                |
|-------------------|---------------------------------------------|-----------------------------------------------|
| Iterates on       | the value $v$                               | the policy $g$                                |
| Inner step        | a maximization                              | a root-find                                   |
| Converges because | $T$ contracts (Blackwell)                   | $K$ is monotone (Coleman; Greenwood–Huffman)  |
| Handles           | kinks, discrete choice, binding constraints | smooth interior problems, distorted economies |
| Accuracy here     | $\log_{10} \mathcal{E} = -1.8$              | $\log_{10} \mathcal{E} = -3.8$                |

Away from this benchmark there is **no closed form to grade against**. Two methods that rest on different theorems, agreeing to four digits, is then the strongest evidence available — which is why cross-method agreement sits on 1.B2's verification checklist, and why the answer to “which one?” is usually “both, once”.

# Where Both Methods Break

Everything today rested on **one continuous state and one small chain**. Four things break it, and you have already met three of them:

- **A second continuous state.** The grid becomes  $N^2$  and a VFI sweep  $N^4$ . 1.B2 counted where that ends: ten dimensions at a hundred points is  $10^{20}$  grid points.
- **A binding constraint.**  $a' \geq -\underline{a}$  turns the Euler equation into a complementarity condition. The root-find of this deck simply has no root.
- **A distribution as a state.** In 2.B1 the state included  $\Gamma$  — an infinite-dimensional object. There is no grid for it.
- **Patience.** At a quarterly  $\beta = 0.99$ , VFI needs 1,379 sweeps and the error amplifier is 99, not 19.

Not one of these is exotic. The first three are the standard models of Session 2 — Aiyagari, Krusell–Smith, life-cycle — and the fourth is the standard calendar.

So the honest summary of today is: **we can solve the trunk, and only the trunk**. The next two sessions are the tools that get us onto the branches.

# The Techniques You Need, and When We Build Them

| Technique                         | Which of today's gaps it closes                                              |    |
|-----------------------------------|------------------------------------------------------------------------------|----|
| Root-finding                      | opens the brentq box: bisection, secant, Newton, Brent, and their rates      | S4 |
| Optimization                      | replaces the grid search that made the staircase                             | S4 |
| Differentiation                   | Newton needs derivatives; finite differences, then automatic differentiation | S4 |
| Approximation & interpolation     | better than straight lines, without losing shape                             | S4 |
| Quadrature & AR(1) discretization | where $\Pi$ actually comes from                                              | S4 |
| Acceleration                      | cuts the 272 sweeps and the $O(N^2)$ inner loop                              | S4 |
| Perturbation                      | a local expansion that ignores the grid entirely                             | S5 |
| Projection                        | a global basis instead of $N$ separate values                                | S5 |
| Nested fixed points               | Aiyagari solved end to end                                                   | S6 |

Nothing on this list is decoration. **Every row was put there by a specific failure you watched happen this morning.**

# Takeaways

- The Euler equation is a functional equation too. Iterating on the **policy** means solving one scalar equation per state instead of searching for a maximum.
- The economics does the numerical work: the residual is increasing on one side and decreasing on the other, so the root exists, is unique, and a bracketing solver **cannot** fail. That argument is what licenses the black box.
- Time iteration converges by **monotonicity**, not contraction. **Coleman** proves it in the planner's world; **Greenwood–Huffman** carry it to where no planner exists — taxes, externalities, distortions.
- Measured on the same grid, and graded on the same object: 17 sweeps against 272, consumption error  $6.7 \times 10^{-6}$  against  $5.4 \times 10^{-3}$ ,  $\log_{10} \mathcal{E}$  of  $-3.8$  against  $-1.8$ . Fewer iterations, better answers, more wall time — each sweep does 400 root-finds.
- Both methods stop at **one continuous state**. A second state, a binding constraint, or a distribution as a state, and today's toolkit is finished.

# Bridge: One Worked Example, Then the Machine Itself

Every claim this morning was a measurement, and every measurement came out of twenty to forty lines of Python. That is not incidental to the argument — it *is* the argument. A method you cannot implement is a method you cannot check.

So Block B is the other half of the bridge from paper to machine: the environment, the language, and the one idea that turns an algorithm on a whiteboard into an array operation that runs in milliseconds.

## Next, then after the break

**3.A4** The RBC model with depreciation, solved line by line — every number on the slide out of the code.

**3.B1** Your toolkit: Python, environments, notebooks, and version control.

**3.B2** From loops to arrays — we rebuild this morning's solver as a class, and make it a few hundred times faster without changing a single number.

**3.B3** PyTorch from the ground up, and how to work with an AI assistant without outsourcing your judgment.